

Self Learning 1

1. Comparison of Agile and DevOps

Agile is a software development methodology that focuses on developing software in small, iterative increments. Instead of completing the entire project at once, the work is divided into smaller parts called iterations or sprints. Agile emphasizes customer feedback, teamwork, continuous improvement, and adapting to changing requirements.

DevOps is a methodology and culture that combines Software Development (Dev) and IT Operations (Ops). Its goal is to improve collaboration between development and operations teams and enable faster, more reliable, and automated software delivery

Comparison Dimension	Agile Methodology	DevOps Methodology
Core Philosophy & Focus	Iterative development, adaptability, functional delivery, and customer collaboration during the software build process.	End-to-end integration of Development and Operations; continuous deployment, automated delivery pipelines, and operational stability.
Primary Scope	Focuses on the early and middle phases of the SDLC (Requirements, Design, Development, and Testing).	Spans the entire lifecycle: Development, Build, Test, Release, Deployment, Operations, and Continuous Monitoring.
Team Composition & Structure	Cross-functional units comprising Product Owners, Scrum Masters, Developers, and QA Testers.	Merged cross-domain engineering units involving Developers, QA, Systems Administrators, SREs, and Operations Engineers.

Comparison Dimension	Agile Methodology	DevOps Methodology
Delivery Mechanisms	Delivers software in fixed, predictable time-boxed iterations (Sprints) typically lasting 1 to 4 weeks.	Pushes changes continuously or on-demand via automated CI/CD pipelines, often multiple times per day.
Automation Focus	Emphasizes test-driven development (TDD), unit testing automation, and automated functional build verifications.	Extensive focus on Infrastructure as Code (IaC), automated deployment pipelines, release verification, and real-time monitoring.
Feedback Loops	Gathers feedback primarily from end-users, Product Owners, and stakeholders during Sprint Reviews and Retrospectives.	Relies on automated real-time telemetry, server health metrics, log analysis, crash reports, and user usage tracking.
Core Metrics	Velocity, Sprint Burndown, Lead Time for Features, Story Point Completion Rate, Defect Density per Sprint.	Deployment Frequency, Lead Time for Changes, Mean Time to Detect (MTTD), Mean Time to Recover (MTTR), Change Failure Rate.
Key Toolstacks	JIRA, Trello, VersionOne, Rally, Confluence, Miro, Azure Boards.	Jenkins, GitLab CI, GitHub Actions, Docker, Kubernetes, Terraform, Ansible,

2. Case Study of JIRA and Asana in Real Projects

2.1 Case Study: JIRA in a Software Development Project

Consider a company developing an **E-Commerce Web Application**. The project includes features such as user registration, product management, shopping cart, payment processing, and order tracking. The development team consists of a project manager, developers, testers, UI/UX designers, and product owners.

The team follows the **Scrum methodology** and uses Jira to manage the software development lifecycle. The complete project is divided into **Epics, User Stories, Tasks, and Bugs**.

For example:

- **Epic:** Online Shopping System
- **User Story:** As a customer, I want to add products to my cart so that I can purchase multiple products together.
- **Task:** Design the shopping cart database schema.
- **Task:** Develop the Add-to-Cart API.
- **Task:** Create the shopping cart user interface.
- **Bug:** Cart total is calculated incorrectly when a discount coupon is applied.

The Product Owner prioritizes items in the **Product Backlog**. During Sprint Planning, the development team selects the highest-priority user stories and moves them into a Sprint. Developers update the status of their work through stages such as **To Do → In Progress → Code Review → Testing → Done**.

Jira provides boards, backlogs, customizable workflows, dashboards, and reporting capabilities that help teams track the progress of work. Its ability to manage dependencies, technical issues, and software development workflows makes it particularly suitable for complex engineering projects.

A real example of Jira being used for large-scale collaboration comes from Atlassian itself, where more than 150 people across 20 teams and six time zones coordinated a major Jira product initiative within three months. Teams used shared goals, backlogs, Agile practices, and different project views to coordinate product, engineering, marketing, and sales activities.

Result: JIRA provides strong traceability, visibility, and control over software development activities. It is especially useful when a project contains many user stories, bugs, releases,

dependencies, and multiple development teams.

2.2 Case Study: Asana in a Cross-Functional Project

Consider a company launching a **new mobile application**. The project requires collaboration between the marketing, design, content, business, and development teams. Unlike a purely technical software project, the work includes activities such as designing promotional material, preparing social media campaigns, creating documentation, conducting market research, and coordinating the product launch.

The company uses Asana to organize and monitor these activities. The project manager creates a project and divides it into sections such as:

1. Planning
2. UI/UX Design
3. Development
4. Testing
5. Marketing
6. Product Launch

Each activity is created as a task and assigned to a responsible team member. Deadlines, priorities, dependencies, comments, and attachments can be added to tasks. Team members can view the project using different views such as lists, boards, timelines, or calendars, depending on the team's workflow.

For example:

Task	Assigned To	Dependency
Finalize application design	UI/UX Designer	None
Develop application	Development Team	Design completion
Perform testing	QA Team	Development completion

Create marketing campaign	Marketing Team	Product information
Launch application	Project Manager	Testing and marketing completion

A practical cross-functional scenario is supported by Asana's Jira integration. For example, a marketer can track a product launch in Asana while engineers continue managing development work in Jira; task and issue synchronization can help both groups follow relevant progress without leaving their primary tool.

Result: Asana is particularly useful for coordinating non-technical and cross-functional work where simplicity, task ownership, deadlines, and collaboration are the primary requirements.

3. Writing Effective User Stories and Acceptance Criteria

3.1 User Stories

A user story is a short description of a requirement written from the user's point of view. It explains who needs a feature, what they need, and why they need it.

Basic Format:

As a [user], I want [feature], so that [benefit].

Example:

As a customer, I want to add products to my shopping cart so that I can purchase them later.

Characteristics of an Effective User Story

- Clear and easy to understand
- Focused on the user's needs
- Provides business or user value
- Small enough to implement
- Can be tested
- Allows discussion between the team and stakeholders

3.2 Acceptance Criteria

Acceptance criteria are the conditions that a feature must satisfy to be considered complete and accepted by the customer or product owner.

Example

User Story: As a customer, I want to add a product to my shopping cart so that I can purchase it later.

Acceptance Criteria:

- The customer can click the Add to Cart button.
- The selected product appears in the shopping cart.
- The cart quantity is updated when the same product is added again.
- The total price is updated automatically.
- The customer receives a confirmation that the product was added successfully.

Importance of Acceptance Criteria

Acceptance criteria make requirements clear, measurable, and testable. They also help developers understand what needs to be implemented and help testers determine whether the feature works correctly.

4. Advanced Requirement Elicitation Techniques

Requirement elicitation is the process of collecting and understanding the needs, expectations, and requirements of stakeholders before developing a software system. Advanced techniques help identify detailed and sometimes hidden requirements.

4.1 Interviews

An interview is a direct discussion between a requirements analyst and stakeholders to understand their needs, problems, and expectations.

Types

- Structured: Uses a fixed set of questions.
- Semi-structured: Uses prepared questions with flexibility for additional questions.
- Unstructured: Allows an open-ended discussion.

Example

For a hospital management system, an analyst may interview doctors, nurses, and receptionists to understand their daily tasks and system requirements.

Advantages

- Provides detailed information.
- Allows follow-up questions.

- Helps understand stakeholder expectations.

4.2 Ethnography

Ethnography is a technique in which the analyst observes users in their actual working environment to understand how they perform their tasks. Instead of only asking users what they do, the analyst observes their real activities and identifies requirements that users may not mention during interviews.

Example

For a warehouse management system, an analyst may observe workers while they scan products, update inventory, and move packages. This can reveal requirements such as mobile access, barcode scanning, and support for poor network connectivity.

Advantages

- Identifies hidden requirements.
- Provides understanding of real-world workflows.
- Captures actual user behavior.
- Useful for complex working environments.

Interviews vs Ethnography

Aspect	Interviews	Ethnography
Method	Collects requirements by directly asking stakeholders questions.	Collects requirements by observing users in their workplace.
Understanding	Helps understand what users say they need.	Helps understand what users actually do.
Time	Faster and easier to conduct.	Usually requires more time and observation.
Useful for	Understanding opinions and expectations.	Discovering hidden and practical requirements.

5. Requirement Traceability Matrix (RTM)

A Requirement Traceability Matrix (RTM) is a structured document used to track software requirements throughout the development lifecycle. It links each requirement with its corresponding design, implementation, and test cases.

The main purpose of RTM is to ensure that all requirements are implemented and tested and that no requirement is missed.

Requirement ID	Requirement	Design	Test Case	Status
R01	User login functionality	D01	TC01	Completed
R02	Product search functionality	D02	TC02	Completed
R03	Online payment functionality	D03	TC03	In Progress
R04	Order history functionality	D04	TC04	Not Started

Benefits of RTM

- Ensures complete requirement coverage.
- Helps identify missing requirements.
- Connects requirements with test cases.
- Makes requirement changes easier to track.
- Improves project quality and documentation.

6. Requirement Management Tools – IBM DOORS

IBM DOORS (Dynamic Object-Oriented Requirements System) is a requirement management tool used to manage requirements in complex and large-scale projects. It helps teams document, organize, trace, review, and control requirements throughout the software development lifecycle.

Key Features of IBM DOORS

Feature	Description
Requirement Management	Allows requirements to be created, organized, and maintained.
Traceability	Links requirements with design, development, and testing activities.
Change Management	Tracks changes and helps identify their impact.

Version Control	Maintains the history of requirement modifications.
Collaboration	Allows stakeholders to review and discuss requirements.
Reporting	Provides structured requirement documents and reports.

Example: IBM DOORS in an Automotive Braking System

Consider an automotive braking system project. One of the system requirements may be:

Requirement: “The braking system shall respond within the specified time after the driver applies the brake pedal.”

This requirement can be recorded and managed in IBM DOORS. It can then be linked throughout the development lifecycle as follows:

Requirement → Design → Implementation → Test Case → Test Result

For example:

- Requirement: The braking system must respond within the specified time.
- Design: Design of the Electronic Brake Control Unit (EBCU).
- Implementation: Development of the braking control software.
- Test Case: Verify whether the braking system responds within the required time.
- Test Result: Pass or Fail based on the actual response time.

If the original requirement is changed, for example, by reducing the allowed response time, IBM DOORS can help the team perform impact analysis. The traceability links allow engineers to identify which design components, software modules, and test cases may be affected by the change.

Thus, IBM DOORS provides end-to-end requirement traceability, ensuring that every requirement can be tracked from its initial definition through design, implementation, and testing.